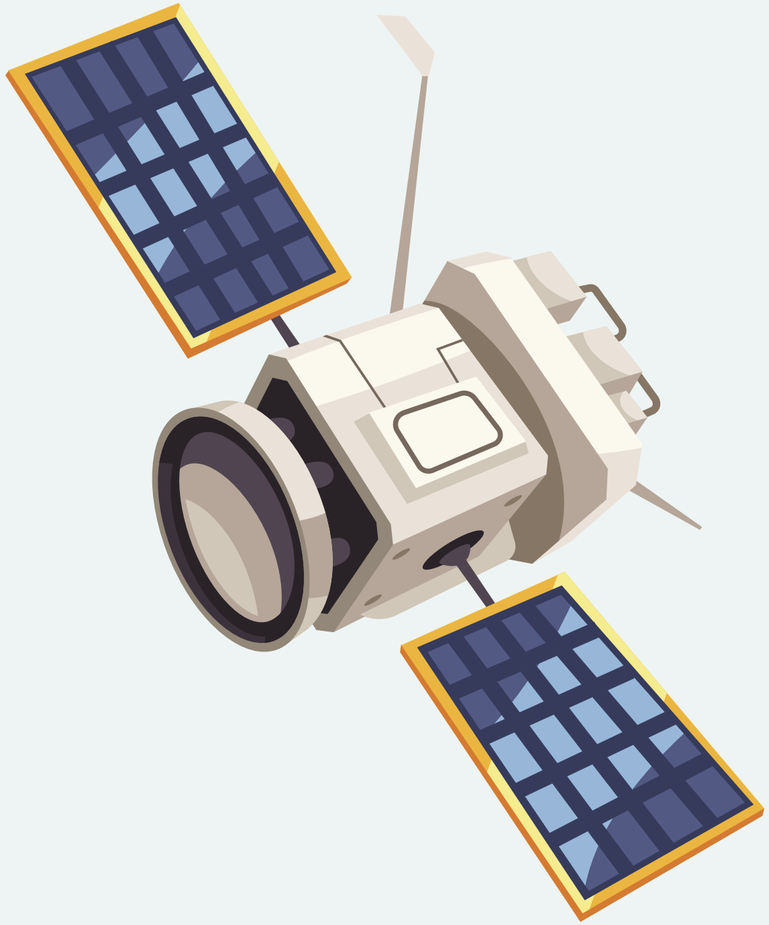
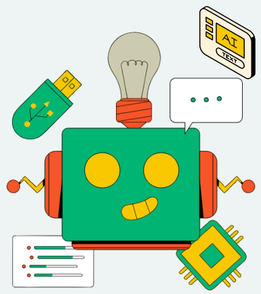


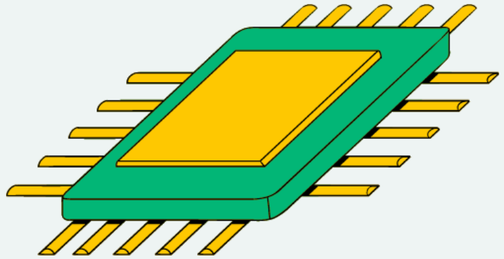
VISION BASED POSE ESTIMATION FOR NON-COOPERATIVE DOCKING/INSPECTION SATELLITE

PRESENTATION

PRESENTED BY:

MAYANK YADAV





SATELLITE POSE ESTIMATION DATASET

We have two main datasets:

1. **SPEED** :

It contains 12000 synthetic images and 5 real images with pose label in quaternion form . Images are 8 bit monochrome in jpeg format, with a resolution of 1920×1200 pixels. Note that the images in the Test split don't have their corresponding labels.

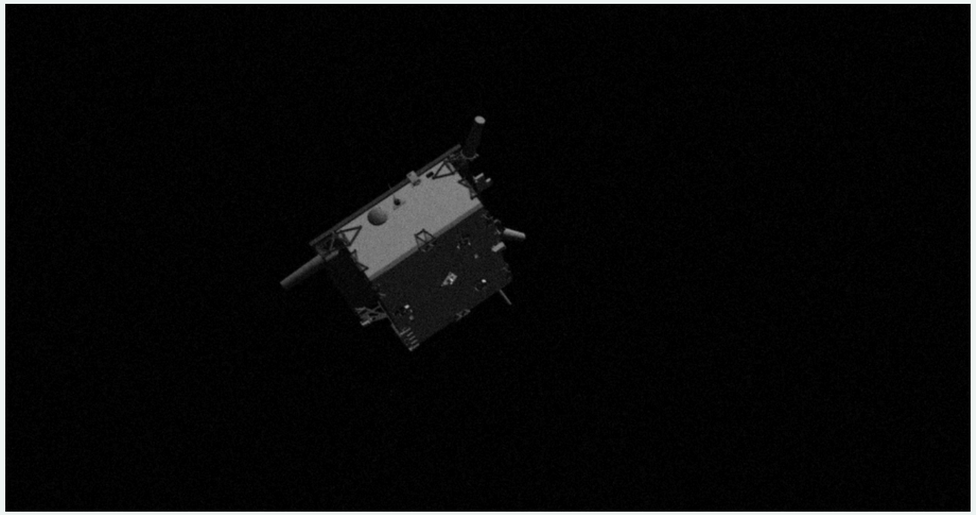
2. **SPEED Plus** :

It is similar to SPEED dataset but have more number of images and have more real images with labels.

	SPEED		SPEED+		
splits	synthetic	real	synthetic	lightbox	sunlamp
Train	12000	5	47966	-	-
Validation	-	-	11994	-	-
Test	2998	300	-	6740	2791

DATASET

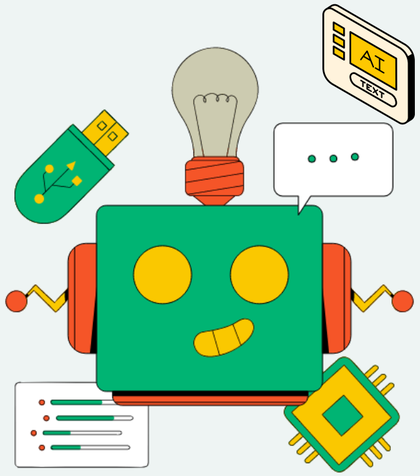
Image

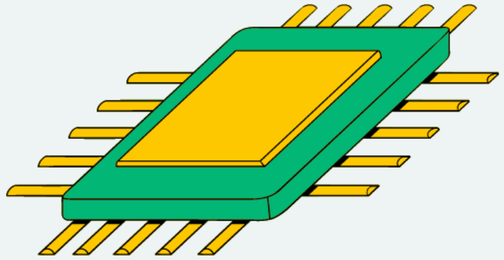


Pose Label

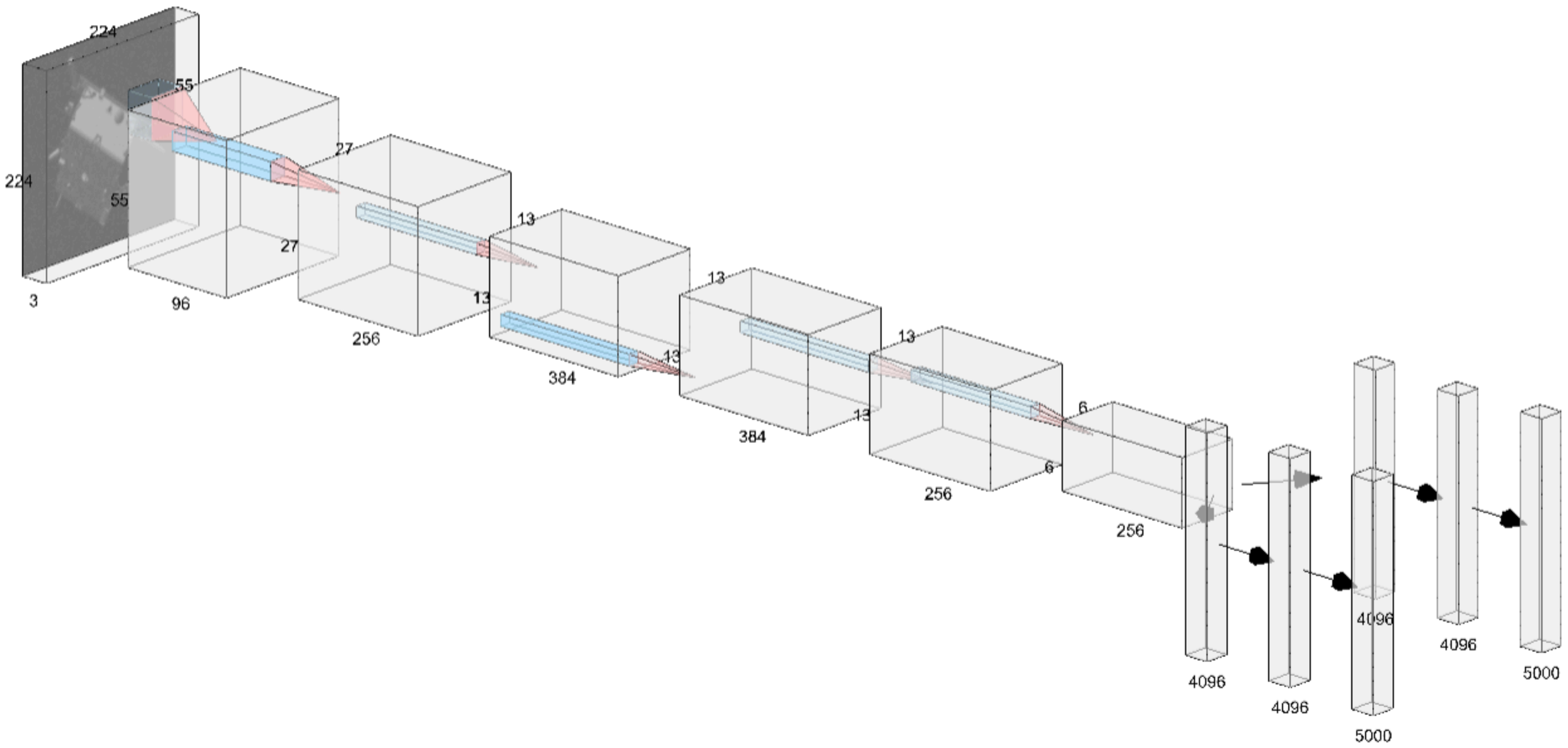

```
0 : {  3 items
  "filename" : string "img000187real.jpg"
  "q_vbs2tango" : [  4 items
    0 : float 0.135249
    1 : float 0.837208
    2 : float -0.350395
    3 : float -0.39751
  ]
  "r_Vo2To_vbs_true" : [  3 items
    0 : float 0.206067
    1 : float 0.268575
    2 : float 3.29739
  ]
}
```


SPACECRAFT POSE NETWORK (SPN)





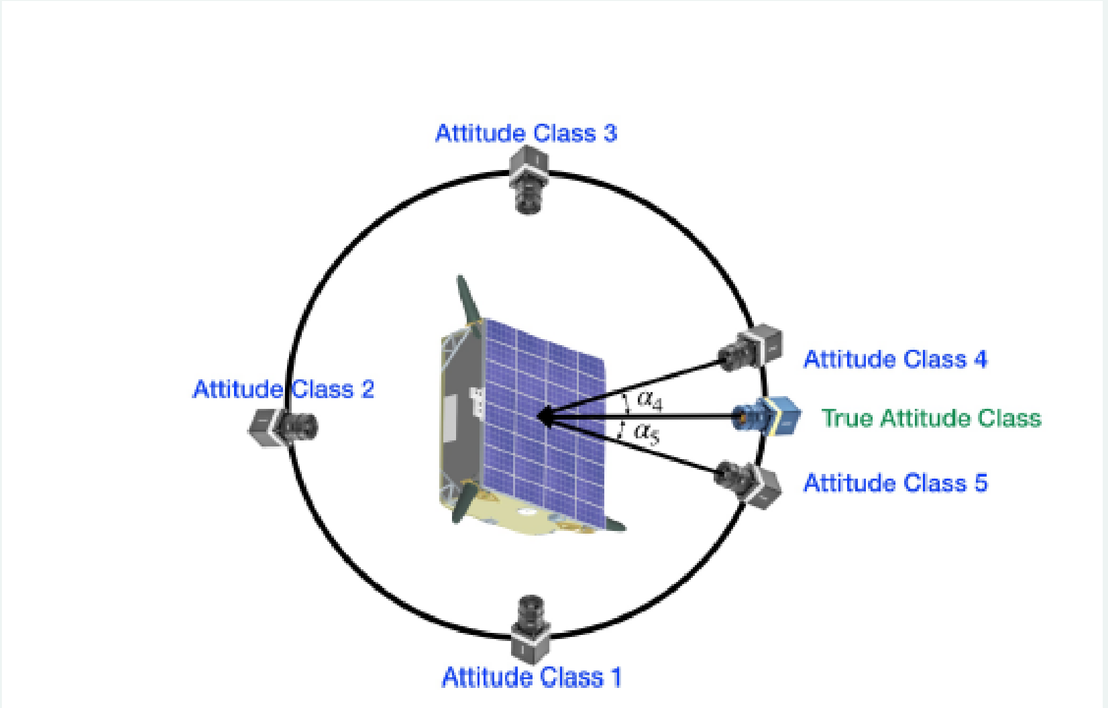
SPN MODEL


224
224
3
55
55
96
27
27
256
13
13
384
13
13
384
13
13
256
6
6
256
4096
4096
4096
5000
4096
5000

SPN MODEL

This model consists of 5 convolution layers followed by two branches of fully connected layers, one is responsible for attitude classification and other is responsible for attitude regression.

We chose n number of uniformly distributed quaternion classes and take m top most classes out of it.



RESULTS

Results Summary:

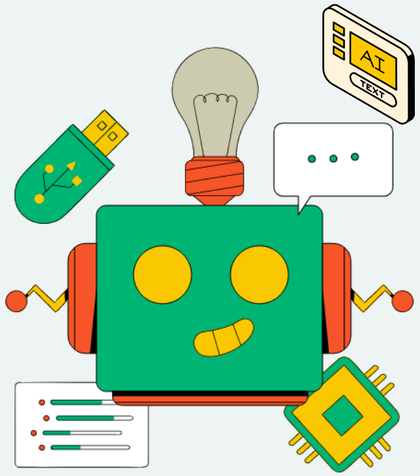
Metric	SPEED synthetic test-set	SPEED real test-set
Mean IoU (-)	0.8407	0.7659
Median IoU (-)	0.8734	0.8000
Mean ET (m)	[0.005 -0.004 0.867]	[0.011 0.109 0.592]
Mean ET mag (m)	0.8668	0.6019
Median ET mag (m)	0.809	0.551
Mean ER (deg)	14.7462	78.8189
Median ER (deg)	6.0831	68.2189

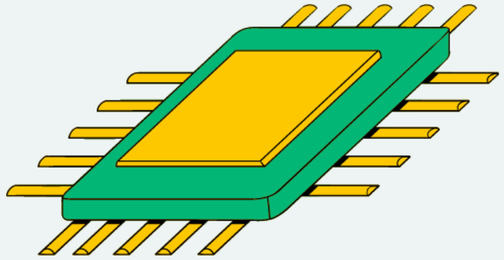
BOTTLENECK

The model performs good on the synthetic test set but bad on real test set because of lack of real images for training. It is unable to generalise the training for real images.

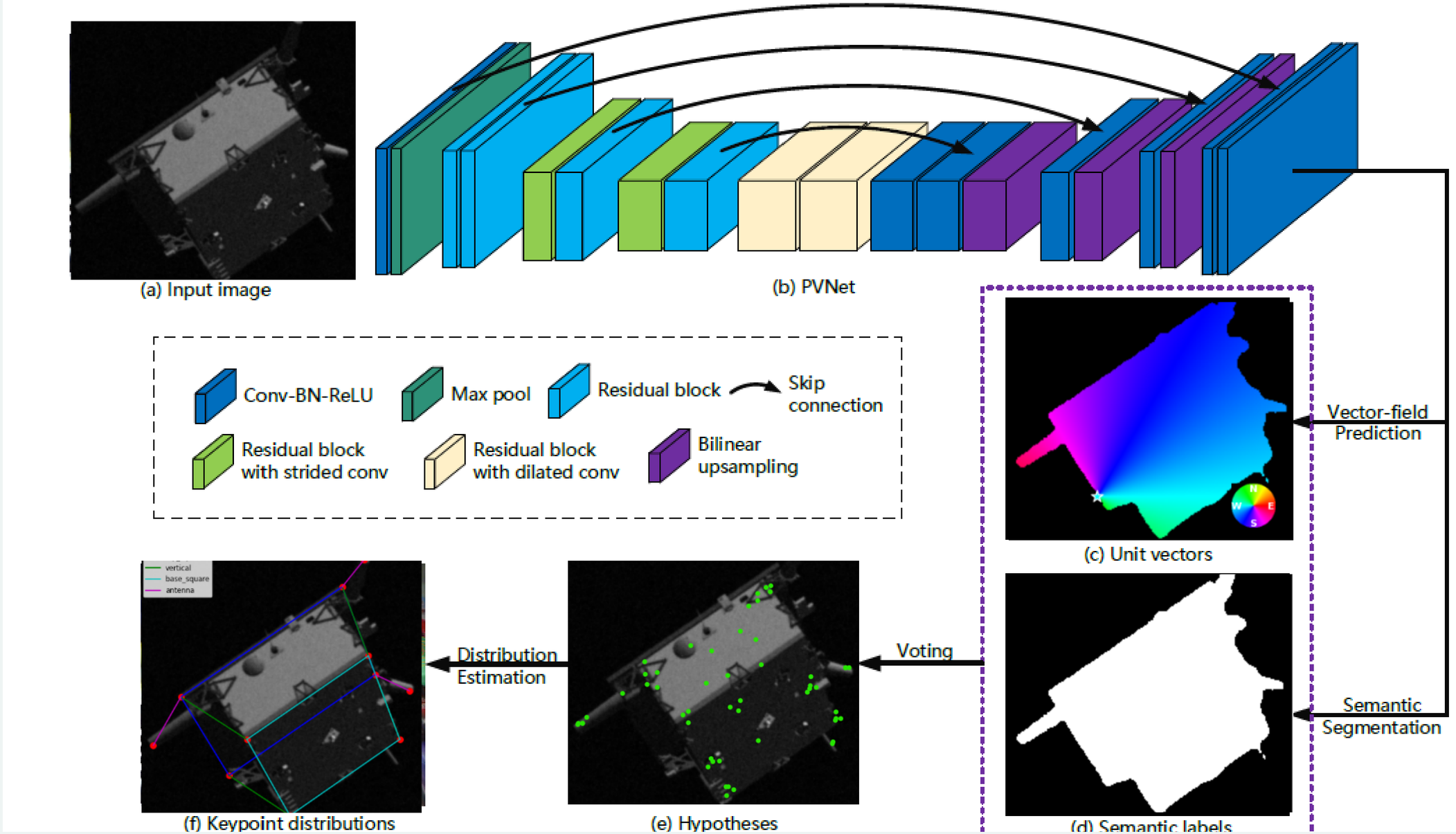
The model can be made better by using the model used by the team that won the first position in Kelvin's Pose Estimation Challenge. They used HRnet. (https://arxiv.org/abs/1908.11542)

PIXEL-WISE VOTING NETWORK FOR 6DOF POSE ESTIMATION (PVNET)





PVNET MODEL


(a) Input image
(b) PVNet
Conv-BN-ReLU
Max pool
Residual block
Skip connection
Residual block with strided conv
Residual block with dilated conv
Bilinear upsampling
Vector-field Prediction
(c) Unit vectors
Semantic Segmentation
(d) Semantic labels
Voting
(e) Hypotheses
Distribution Estimation
vertical
base_square
antenna
(f) Keypoint distributions

PVNET MODEL

This model uses resnet-18 to get a feature pyramids and they are step by step scaled up and combined to original image dimensions for obtaining segmentation mask and vectors predictions.

As masks were not available in SPEED and SPEEDPlus datasets, so I used SAM(segment anything model) to segment them.

For SPEED dataset we are using 11 keypoints.

After obtaining segmentation mask and vectors predictions, we use RANSAC voting.

After RANSAC voting, we have obtained the 2D keypoints on the image, now we use pnp to get the R and T.

RANSAC VOTING

The RANSAC procedure is opposite to that of conventional smoothing techniques: Rather than using as much of the data as possible to obtain an initial solution and then attempting to eliminate the invalid data points, RANSAC uses as small an initial data set as feasible and enlarges this set with consistent data when possible.

For example, given the task of fitting an arc of a circle to a set of two-dimensional points, the RANSAC approach would be to select a set of three points (since three points are required to determine a circle), compute the center and radius of the implied circle, and count the number of points that are close enough to that circle to suggest their compatibility with it (i.e., their deviations are small enough to be measurement errors). If there are enough compatible points, RANSAC would employ a smoothing technique such as least squares, to compute an improved estimate for the parameters of the circle now that a set of mutually consistent points has been identified.

RANSAC VOTING IN PVNET

1. We find the pixels of the target object using semantic labels.
2. We randomly choose two pixels and take the intersection of their vectors as a hypothesis h_(k,i) for the keypoint x_k.
3. This step is repeated N times to generate a set of hypotheses(N hypthesis for each keypoint) that represent possible keypoint locations.
4. Finally, all pixels of the object vote for these hypotheses. With voting score, calculated in the following way:

$$w_{k,i} = \sum_{\mathbf{p} \in O} \mathbb{I}\left(\frac{(\mathbf{h}_{k,i} - \mathbf{p})^T}{\|\mathbf{h}_{k,i} - \mathbf{p}\|_2} \mathbf{v}_k(\mathbf{p}) \geq \theta \right)$$

That is for a hypothesis, we measure the cosine similarity from every pixel of the object O, using vector predictions and the unit vector between hypthesis and the pixel. And if it is above a threshold we add 1.

ITERATIVE PNP (CV2.SOLVEPNP_ITERATIVE)

The SOLVEPNP_ITERATIVE method is based on Levenberg–Marquardt optimization(it blends gradient descent and the Gauss–Newton method.). This iterative approach refines the camera pose estimate by minimizing the reprojection error – the sum of squared distances between the observed 2D image points and the projected 3D object points

RESULTS

PVNET TRAINED ON SPEED

Results Summary:

Metric	SPEED synthetic test-set	SPEED real test-set
Mean ET (m)	[0.071 -0.050 9.845]	[-0.070 -0.128 0.956]
Mean ET mag (m)	9.8451	0.9675
Median ET mag (m)	8.647	0.723
Mean ER (deg)	45.6121	13.3741
Median ER (deg)	12.0449	12.2134

PVNET TRAINED ON SPEED AND SPEEDPLUS

Results Summary:

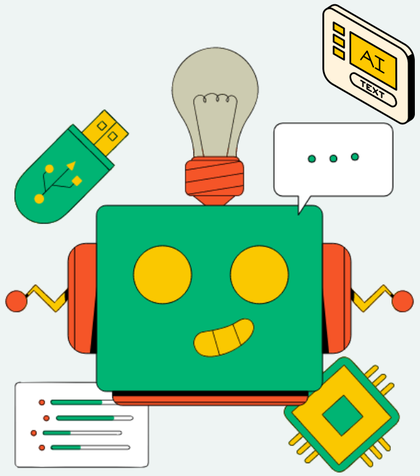
Metric	SPEED synthetic test-set	SPEED real test-set
Mean ET (m)	[0.067 -0.069 9.418]	[-0.150 -0.116 1.277]
Mean ET mag (m)	9.4189	1.2906
Median ET mag (m)	8.151	1.461
Mean ER (deg)	30.9939	7.4885
Median ER (deg)	10.7517	7.9267

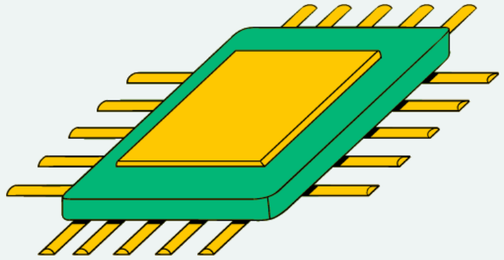
BOTTLENECK

The original PVNet paper had tried various keypoints sampling including FPS(Farthest Point Sampling) and Box corners. But we are using only 11 keypoints.

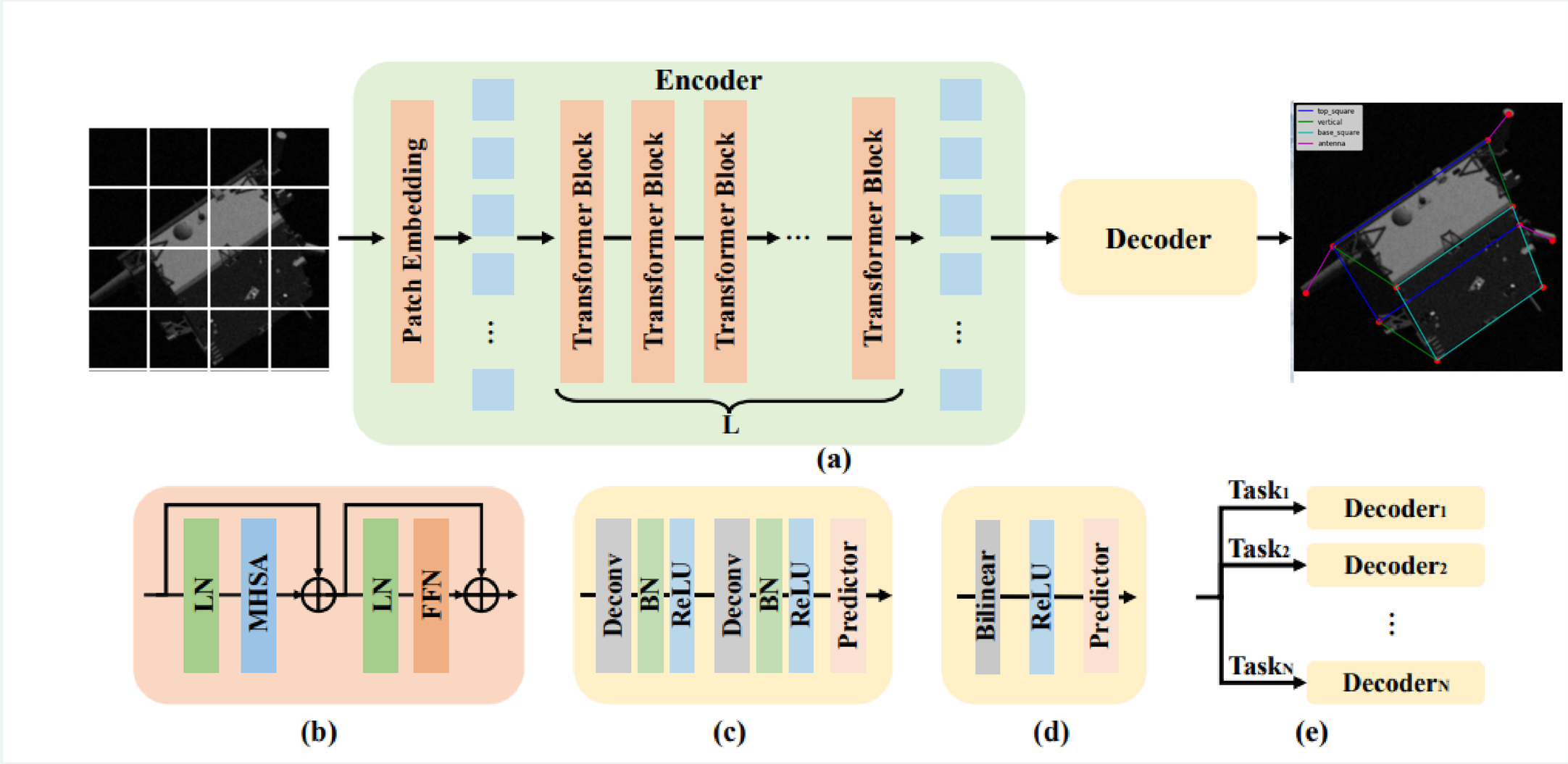
We have used SAM (ViT-H) but some masks are not accurate(it fails to segment antennas in some images).

SIMPLE VISION TRANSFORMER BASELINES FOR HUMAN POSE ESTIMATION (VITPOSE)





VITPOSE MODEL



(A) THE FRAMEWORK OF VITPOSE. (B) THE TRANSFORMER BLOCK. (C) THE CLASSIC DECODER. (D) THE SIMPLE DECODER. (E) THE DECODERS FOR MULTIPLE DATASETS.

VIT MODEL

This model is similar to PVNet in terms of input and output and the processes after predictions. Just Resnet is replaced with ViT for feature extraction and I tried with 3 decoders(classical, simple and PVNet style decoder)

RESULTS:

Results Summary:

Metric	SPEED synthetic test-set	SPEED real test-set
Mean ET (m)	[-0.004 -0.013 10.378]	[-0.127 0.113 4.333]
Mean ET mag (m)	10.3777	4.3367
Median ET mag (m)	8.783	4.150
Mean ER (deg)	117.0704	148.1685
Median ER (deg)	124.6000	142.4881

PROBLEMS WITH USING VIT

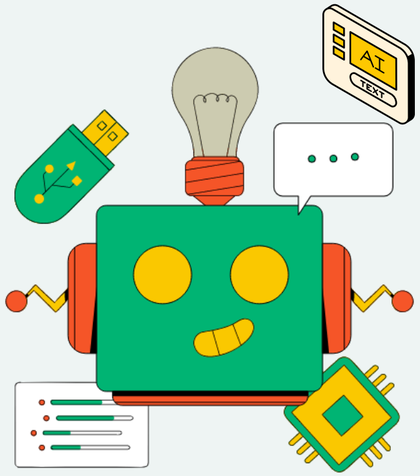
The original ViT Pose paper objective was to get a (H/4,W/4) image from ViT and then use decoder to get 17 keypoints .

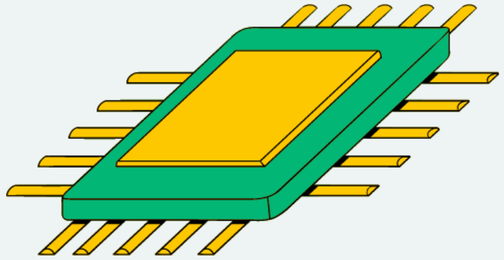
But I require more dense information(for each pixel, a segmentation output and vector prediction for 2xn layers where n is the number of keypoints) on (H,W) image.

Moreover the original PVNet extracted feature pyramids but ViT has the same dimensions of features across depth.

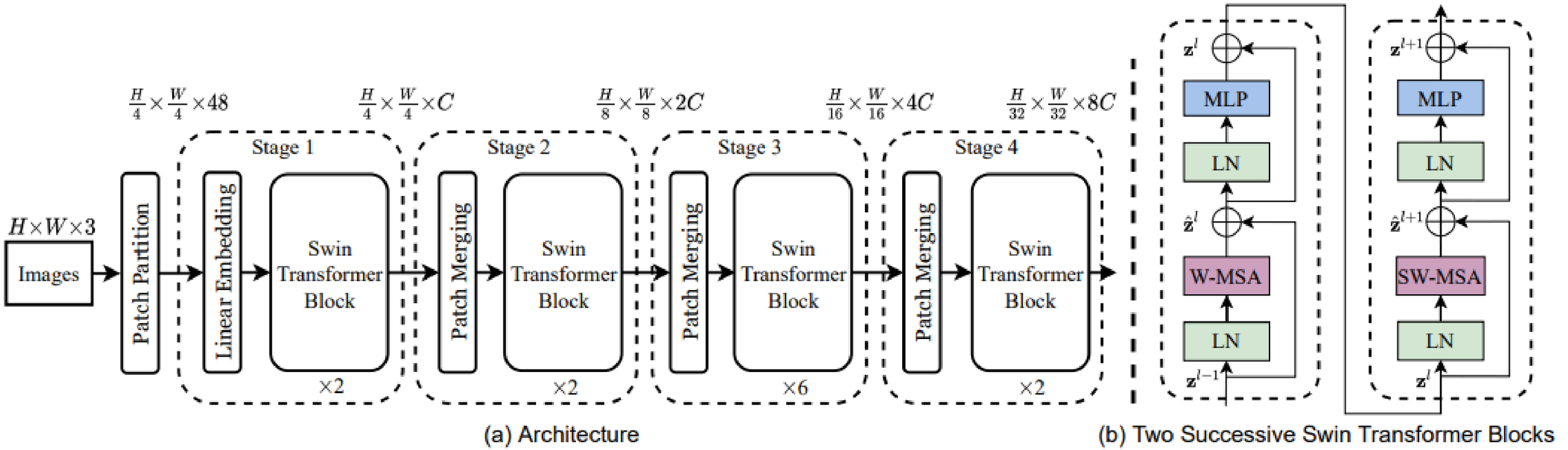
SoI tried using SWIN instead of ViT.

SWIN POSE





SWIN MODEL



(a) Architecture (b) Two Successive Swin Transformer Blocks

Figure 3. (a) The architecture of a Swin Transformer (Swin-T); (b) two successive Swin Transformer Blocks (notation presented with Eq. (3)). W-MSA and SW-MSA are multi-head self attention modules with regular and shifted windowing configurations, respectively.

I tested with two type of decoders:

1.PVNet Decoder:

```python
def upsample_features(self, features):
    """Upsample features from multiple SWIM Stages"""
    f1 = F.interpolate(features[0], scale_factor=2, mode='bilinear', align_corners=False)
    f2 = F.interpolate(features[1], scale_factor=2, mode='bilinear', align_corners=False)
    f3 = F.interpolate(features[2], scale_factor=2, mode='bilinear', align_corners=False)
    f4 = F.interpolate(features[3], scale_factor=4, mode='bilinear', align_corners=False)
    return [f1, f2, f3, f4]
```

```python
        x_feats = self.swin(x)
        # print(f"x_feats shape: {x_feats[0].shape}, {x_feats[1].shape}, {x_feats[2].shape}, {x_feats[3].sh
ape}")

        features = self.upsample_features(x_feats)
        # print(f"Upsampled features shape: {features[0].shape}, {features[1].shape}, {features[2].shape},
{features[3].shape}")
```

```
        x2s = features[0]  #96x128
        x4s = features[1]  #48x64
        x8s = features[2]  #24x32
        xfc = features[3]  #24x32

        # print(f"Before concat xfc: {xfc.shape}, x8s: {x8s.shape}")
        fm = self.conv8s(torch.cat([xfc, x8s], 1))  # 768+384
        fm = self.up8sto4s(fm)

        # print(f"Before concat fm: {fm.shape}, x4s: {x4s.shape}")
        fm = self.conv4s(torch.cat([fm, x4s], 1))   # 128 + 192
        fm = self.up4sto2s(fm)

        # print(f"Before concat fm: {fm.shape}, x2s: {x2s.shape}")
        fm = self.conv2s(torch.cat([fm, x2s], 1))   # 64 + 96
        fm = self.up2storaw(fm)

        # print(f"Before concat fm: {fm.shape}, x: {x.shape}")
        final_out = self.convraw(torch.cat([fm, x], 1))  # 32 + 3

        seg_pred = final_out[:, :self.seg_dim, :, :]
        ver_pred = final_out[:, self.seg_dim:, :, :]
```

RESULTS

Results Summary:

Metric	SPEED synthetic test-set	SPEED real test-set
Mean ET (m)	[0.003 -0.003 10.851]	[-0.306 0.060 3.730]
Mean ET mag (m)	10.8510	3.7427
Median ET mag (m)	9.669	4.224
Mean ER (deg)	87.3644	93.3250
Median ER (deg)	41.6233	115.9585

BOTTLENECK

Transformers require a large number of images and epochs to learn properly.

2. SWIN Segmentation Decoder:

This decoder is based on UPerNet and is used in SWIM Semantic Segmentation implementation.

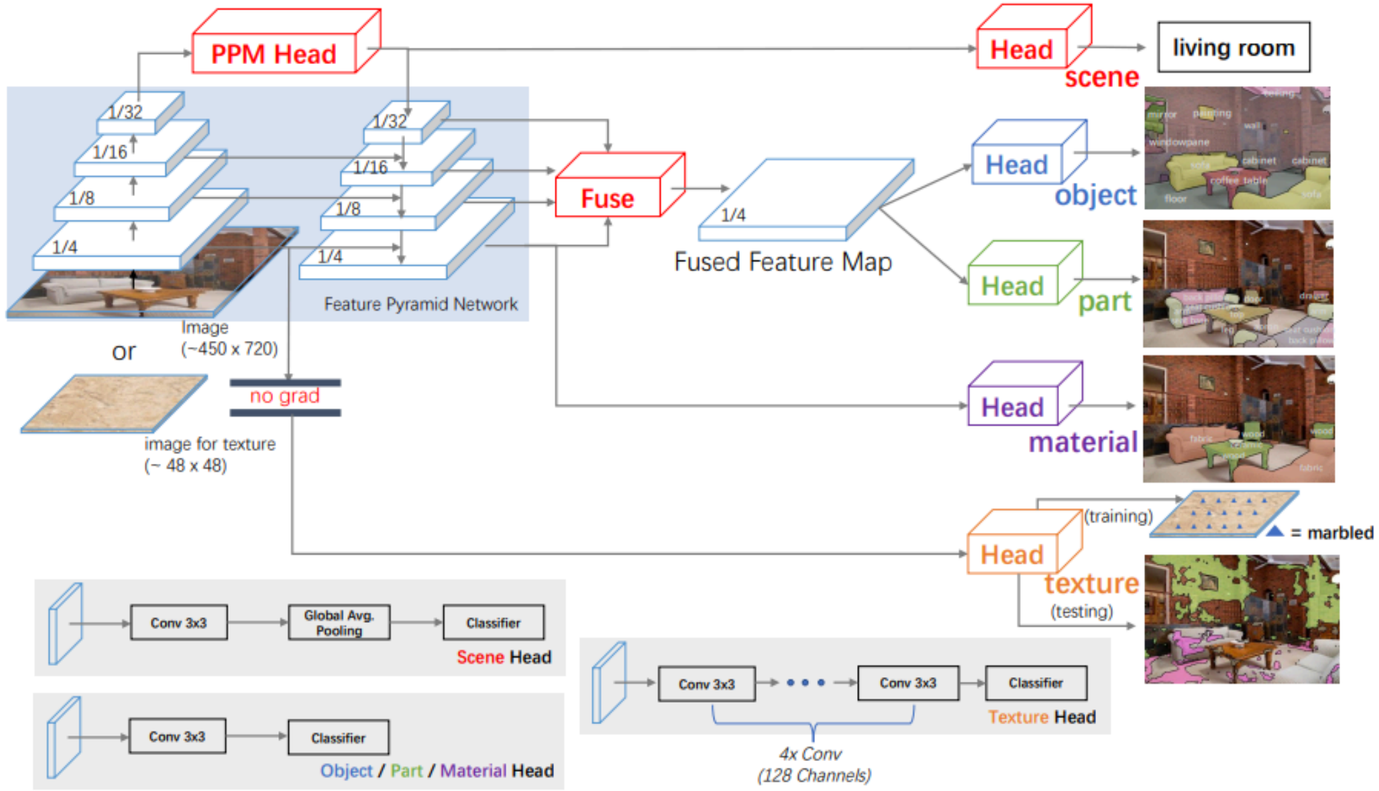


Fig. 4. UPerNet framework for Unified Perceptual Parsing. Top-left: The Feature Pyramid Network (FPN) [31] with a Pyramid Pooling Module (PPM) [16] appended on the last layer of the back-bone network before feeding it into the top-down branch in FPN.

```
self.swin = SwinTransformer(
    pretrain_img_size=(256, 192),
    patch_size=4,
    in_chans=3,
    embed_dim=96,
    depths=[2, 2, 6, 2],
    num_heads=[3, 6, 12, 24],
    window_size=7,
    mlp_ratio=4.,
    qkv_bias=True,
    qk_scale=None,
    drop_rate=0.,
    attn_drop_rate=0.,
    drop_path_rate=0.3,
    norm_layer=nn.LayerNorm,
    ape=False,
    patch_norm=True,
    out_indices=(0, 1, 2, 3),
    frozen_stages=-1,
    use_checkpoint=False
)
```

```
        self.decodeHead = UPerHead(
            in_channels=[96, 192, 384, 768],
            in_index=[0, 1, 2, 3],
            pool_scales=(1, 2, 3, 6),
            channels=512,
            dropout_ratio=0.1,
            num_classes=self.seg_dim + self.ver_dim,
            align_corners=False
        )
```

```
def forward(self, x):
    x_feats = self.swin(x)
    x_decoded = self.decodeHead(x_feats)

    seg_pred = x_decoded[:, :self.seg_dim, :, :]
    ver_pred = x_decoded[:, self.seg_dim:, :, :]

    # print(f"seg_pred shape: {seg_pred.shape}, ver_pred shape: {ver_pred.shape}")

    return seg_pred, ver_pred
```

RESULTS

Results Summary:

Metric	SPEED synthetic test-set	SPEED real test-set
Mean ET (m)	[0.011 -0.010 10.267]	[-0.318 0.263 2.861]
Mean ET mag (m)	10.2669	2.8905
Median ET mag (m)	9.146	3.062
Mean ER (deg)	68.0183	136.6333
Median ER (deg)	24.7395	122.0569

BOTTLENECK

Transformers require a large number of images and epochs to learn properly.